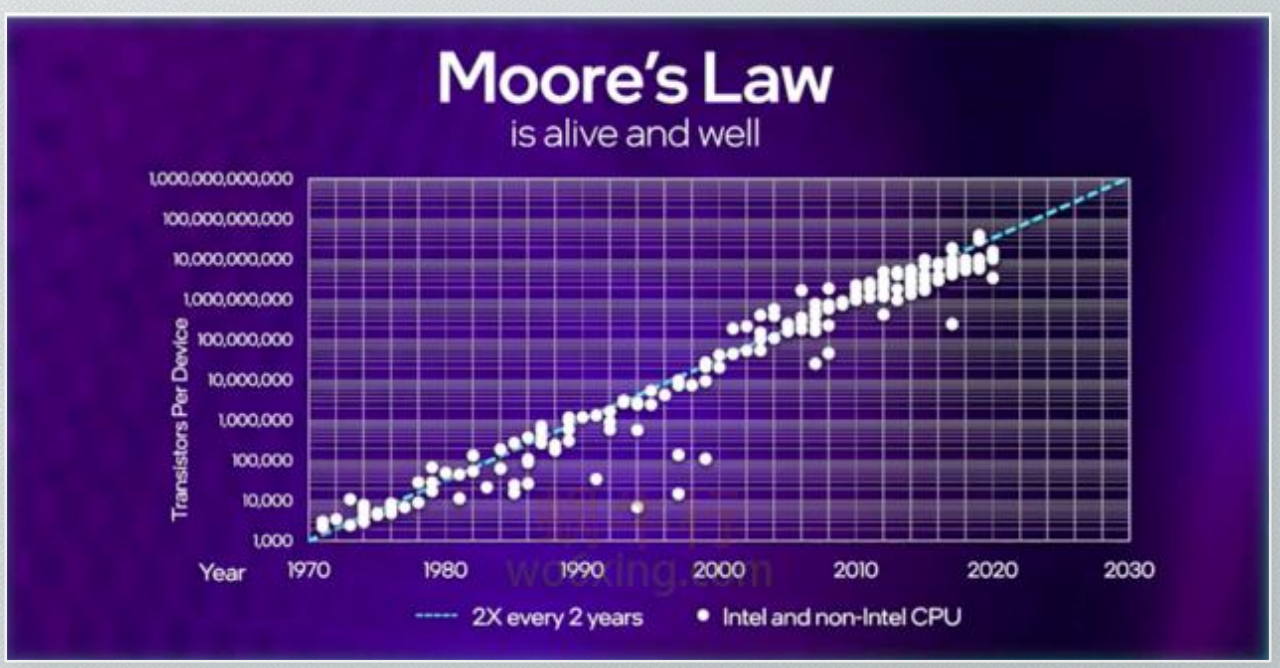
Computer System Architecture

Lecture 1 - Introduction

September 2026

Introduction – The Computer Revolution



Moore's Law: integrated circuit resources double every 18–24 months;
Predicted in 1965 by Gordon Moore, co-founder of Intel;
Result in smaller, faster, and more affordable computing devices.

Introduction – The Computer Revolution



Personal Tech: Smartphones as essential digital hubs.



Scientific Discovery: Supercomputers for climate & genomics.

Computing Systems

Smart Mobility: Microprocessors powering safety & autonomy.



Healthcare: Wearable devices for monitoring & recording.



Goal: To master the fundamental principles governing all computing systems.

Classes of Computing Applications



Personal Computers

Purpose: Designed for individual use, handling everyday tasks.
Traits: Balanced performance, affordability, and broad software compatibility for personal use.



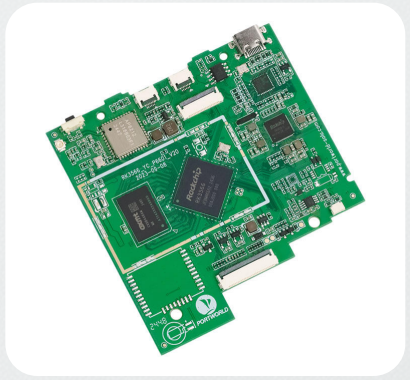
Servers

Purpose: Run larger programs for multiple users, typically accessed via a network.
Traits: Optimized for reliability, scalability, and high throughput to run 24/7 without interruption.



Supercomputers

Purpose: Servers with the highest performance and cost. Tackle complex computational challenges.
Traits: Extreme processing power, the peak of computing capability.



Embedded Computers

Purpose: Designed to run one application, normally integrated with the hardware and delivered as a single system.
Traits: Compact size, low cost and power consumption, tailored for dedicated functions.

The PostPC Era

Personal Mobile Devices (PMDs)

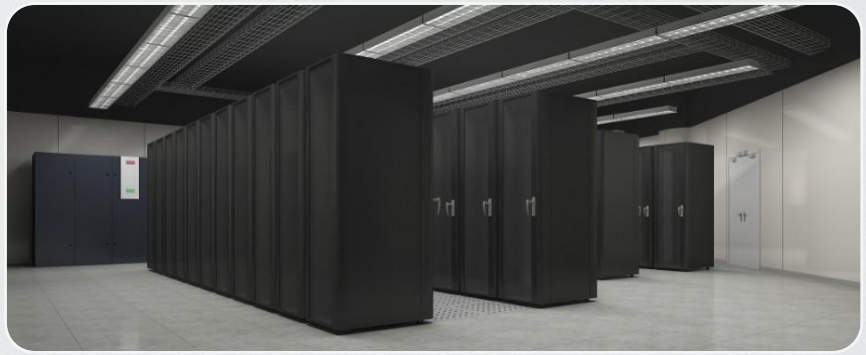




Key Features: Small wireless devices to connect to the Internet; battery-powered and touch-sensitive. Examples: smartphones, tablets, wearables.

Impact: Drives innovation in low-power processors and user-centric design, with billions of units manufactured every year.

Cloud Computing



Key Features: Large collections of servers that provide services over the Internet; rely upon giant datacenters - Warehouse Scale Computers (WSCs).

Impact: Software as a Service (SaaS) deployed via the cloud delivers software and data as a service over the Internet, using a thin program that runs on local client devices.

Great Ideas in Computer Architecture (1/4)

01 Design for Moore's Law

As computer designs can take years, the resources available per chip can easily double or quadruple between the start and finish of the project. Computer architects must anticipate where the technology will be when the design finishes rather than design for where it starts.

02 Use Abstraction to Simplify Design

A major productivity technique for hardware and software is to use abstractions to represent the design at different levels of representation; lower-level details are hidden to offer a simpler model at higher levels.

Great Ideas in Computer Architecture (2/4)

03 Make the Common Case Fast

Making the common case fast will tend to enhance performance better than optimizing the rare case. We focus on optimizing for frequently occurring scenarios to maximize overall efficiency.

04 Performance via Parallelism

Perform operations in parallel to fully utilize hardware resources. This principle leverages multi-core architectures and concurrent processing, turning idle cycles into productive work.

Great Ideas in Computer Architecture (3/4)

05 Performance via Pipelining

Break instruction execution into sequential stages (fetch, decode, execute, write-back) to enable concurrent processing. This parallelism allows multiple instructions to overlap in execution.

06 Performance via Prediction

Speculate on future outcomes (e.g., branch direction or load values) to keep the pipeline filled and avoid costly stalls. Through prediction, processors could maintain high utilization even when facing conditional operations or memory access latency.

Great Ideas in Computer Architecture (4/4)

07 Hierarchy of Memories

Place the fastest, smallest, and most expensive memory (cache) at the top of the hierarchy, and the slowest, largest, and cheapest at the bottom. Create a storage system that optimizes both performance and affordability, ensuring data is readily available when needed.

08 Dependability via Redundancy

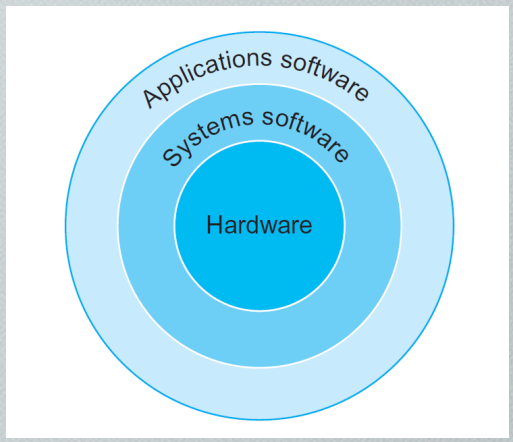
Make systems dependable by including redundant components that can take over when a failure occurs. This fault-tolerant design is essential for mission-critical applications where stable system operation is the key concern.

Below Your Program

01 Application Layer

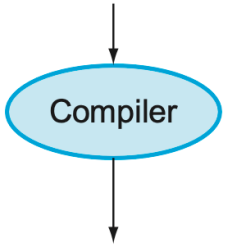
Crafted using **high-level languages** such as Python, Java, and C++.

- Close to human logic
- Concise & improved productivity
- Platform portability



High-level language program (in C)

```
swap(int v[], int k)
{int temp;
   temp = v[k];
   v[k] = v[k+1];
   v[k+1] = temp;
}
```



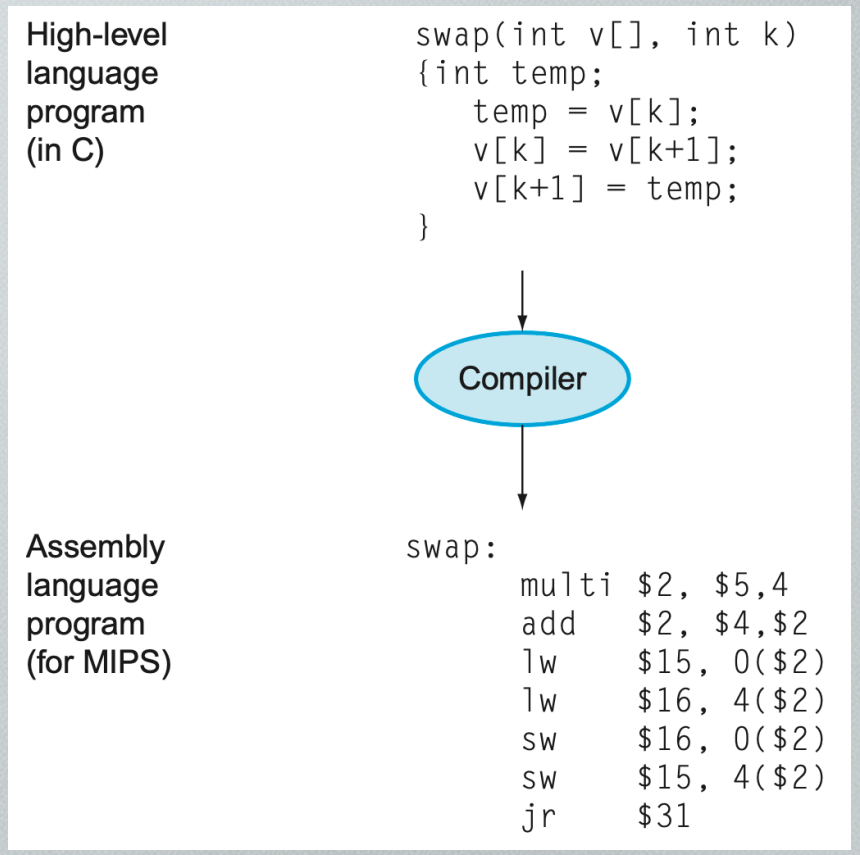
Assembly language program (for MIPS)

```
swap:
      multi $2, $5,4
      add   $2, $4,$2
      lw    $15, 0($2)
      lw    $16, 4($2)
      sw    $16, 0($2)
      sw    $15, 4($2)
      jr    $31
```

Below Your Program

02 System Software

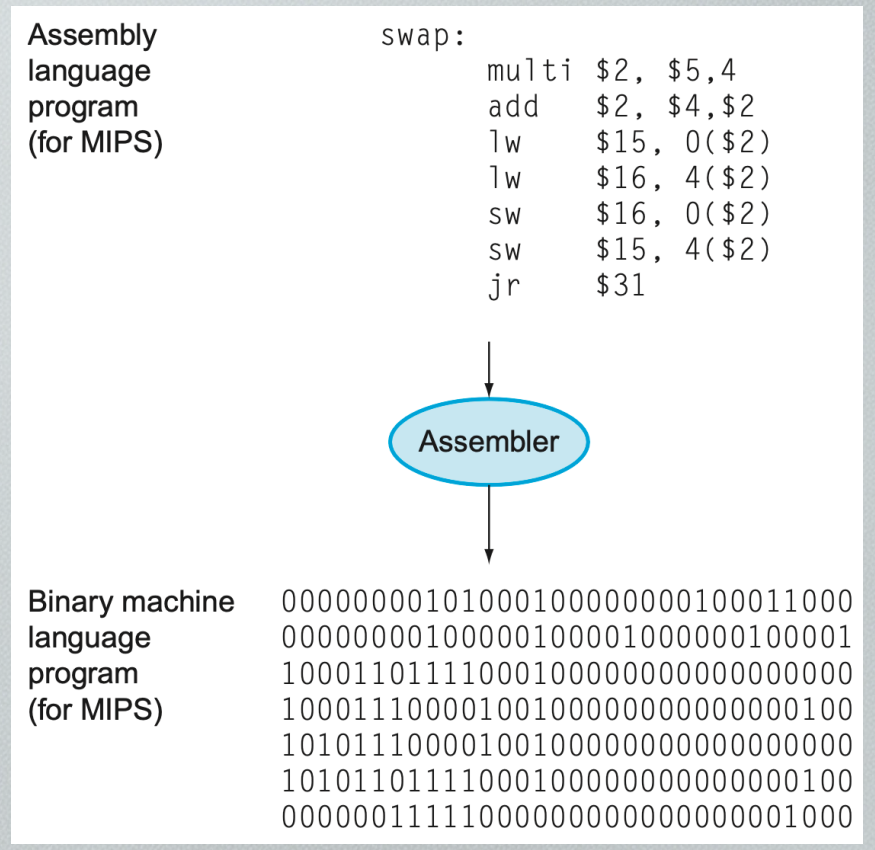
The critical intermediary layer. **Compiler** translates high-level language statements into assembly language statements. **Operating System (OS)** manages hardware resources, memory, I/O operations, and task scheduling for applications.



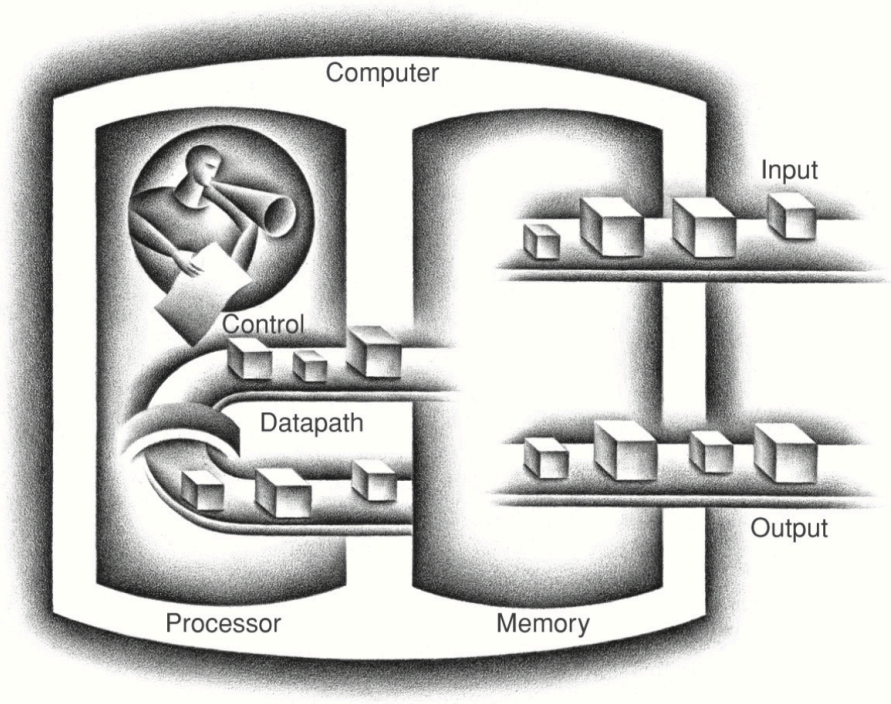
Below Your Program

03 Hardware Layer

Physical foundation of the stack. It executes the binary machine language instructions.
Assembler translates assembly language statements into machine language instructions.



Under the Covers – The Hardware Components



5 classic components of a computer.
The processor (CPU) gets instructions and data from memory. Input writes data to memory, and output reads data from memory. Control sends the signals that determine the operations of the datapath, memory, input, and output.

01. Input

Bring information into the computer (e.g., keyboard).

02. Output

Send computation result out of the computer (e.g., display).

03. Memory

Store programs when they are running and data needed by running programs.

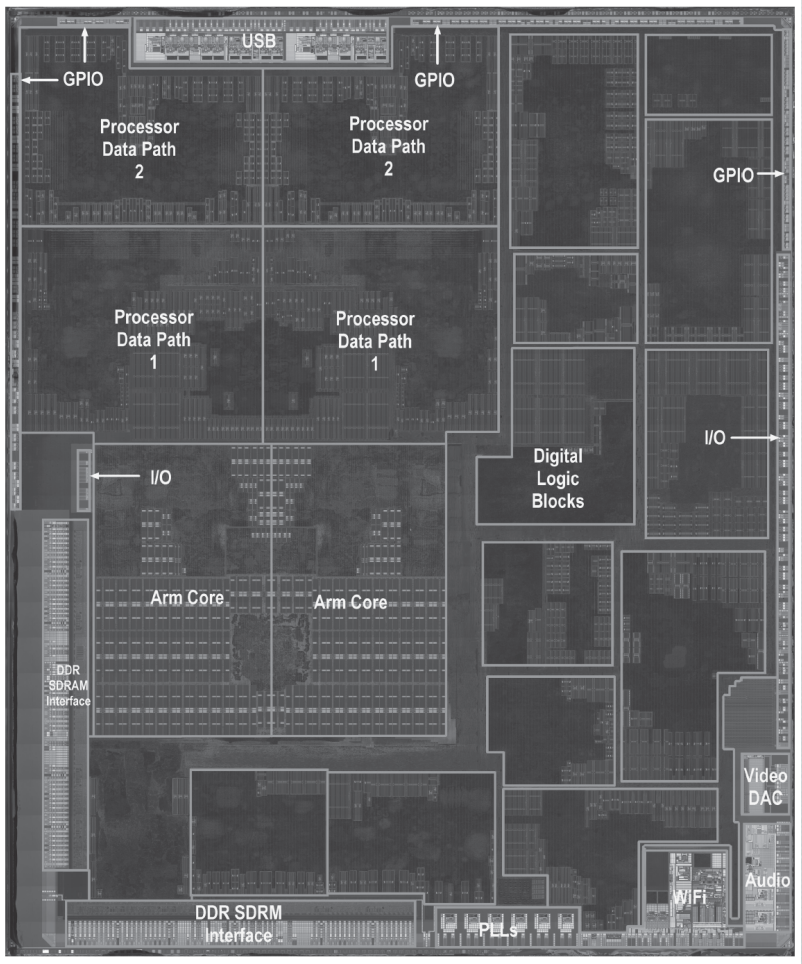
04. Datapath

Component of the processor. Perform arithmetic and logical operations on data (e.g., ALU).

05. Control

Component of the processor. Command the datapath, memory, and I/O devices according to the instructions of the program.

Under the Covers - The Hardware Components



Processor integrated circuit example.

01. Input

Bring information into the computer (e.g., keyboard).

02. Output

Send computation result out of the computer (e.g., display).

03. Memory

Store programs when they are running and data needed by running programs.

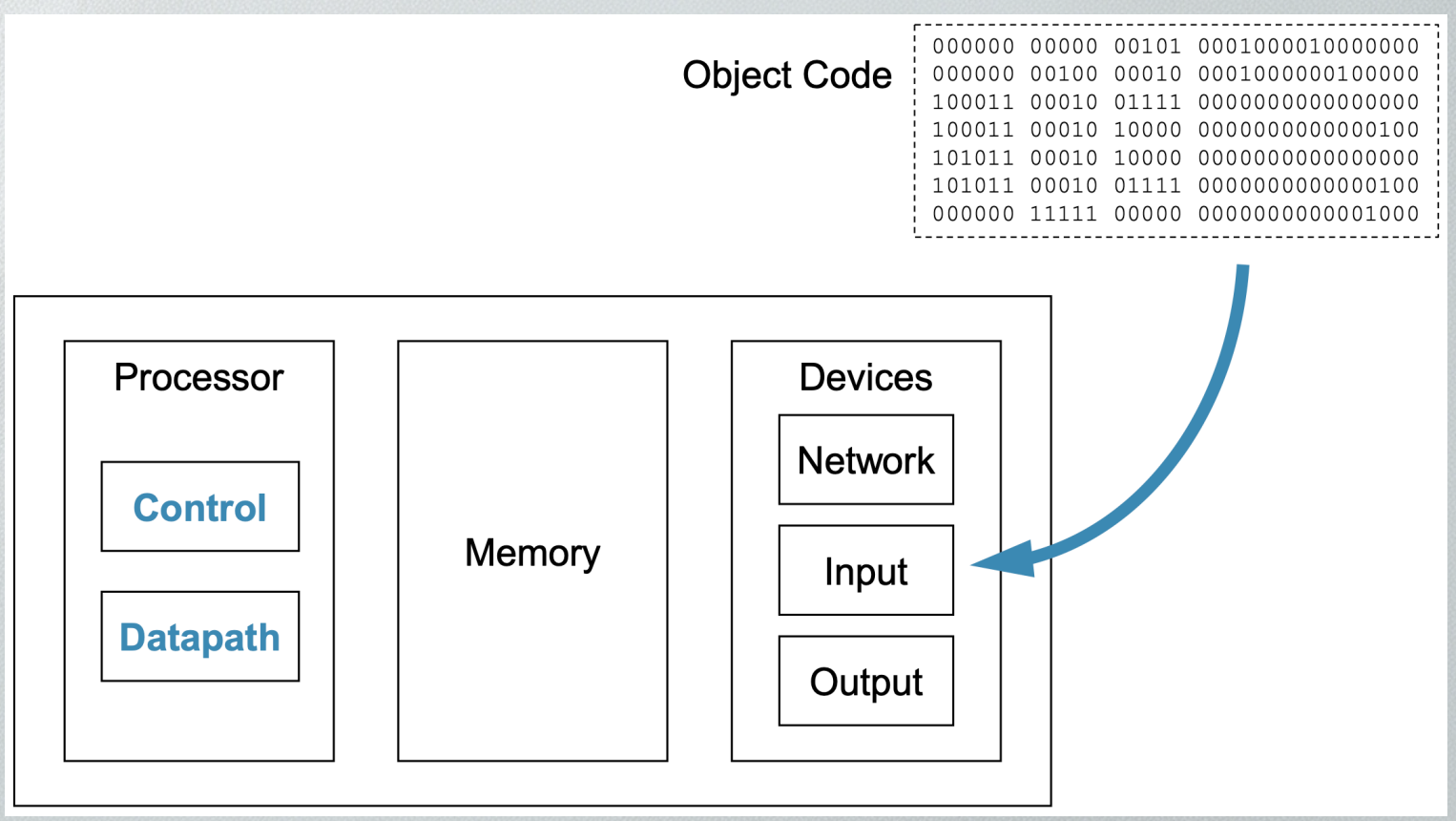
04. Datapath

Component of the processor. Perform arithmetic and logical operations on data (e.g., ALU).

05. Control

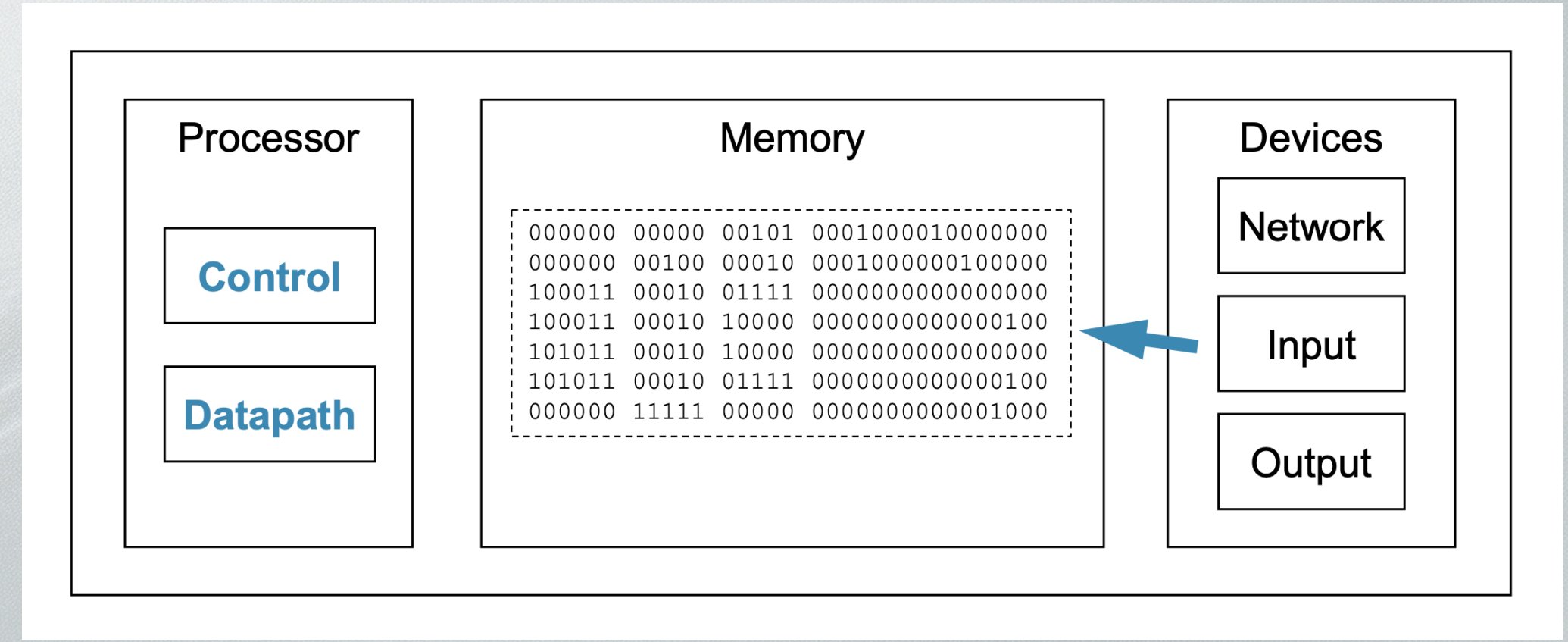
Component of the processor. Command the datapath, memory, and I/O devices according to the instructions of the program.

Under the Covers - The Hardware Components



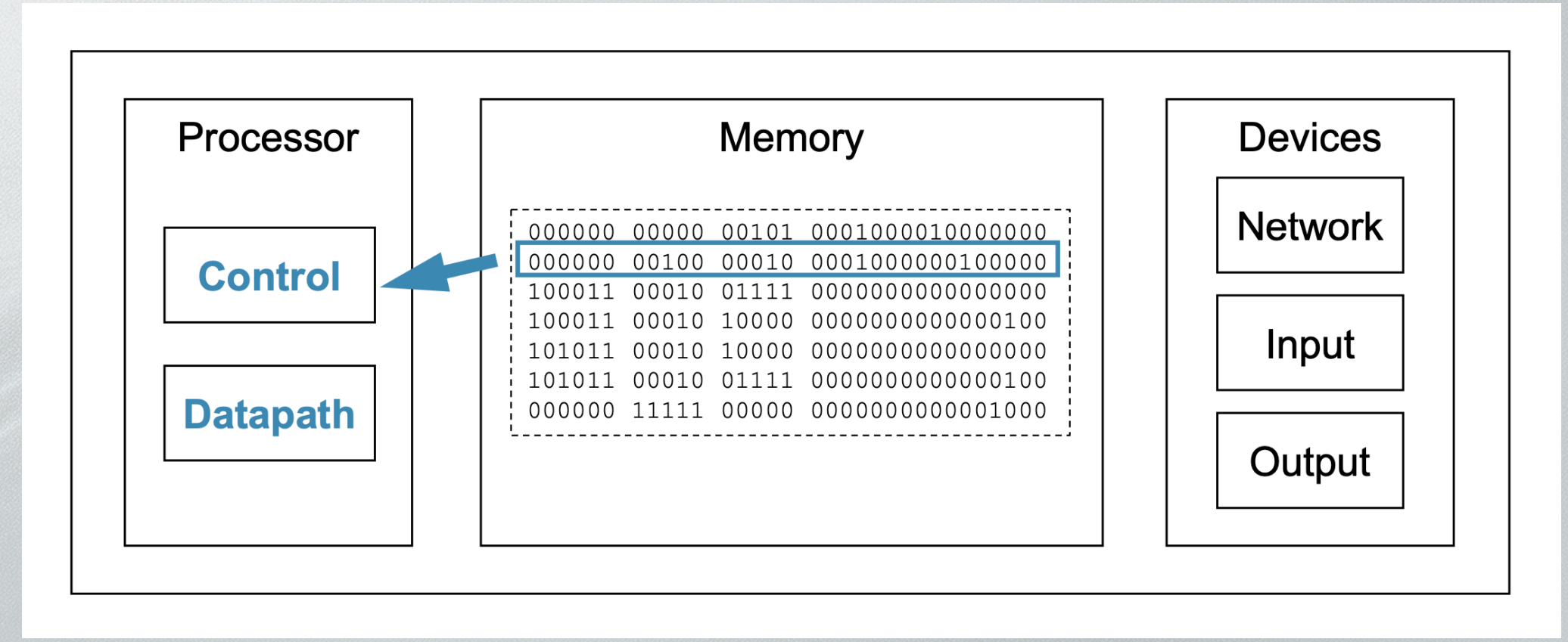
Code Input to Device

Under the Covers - The Hardware Components



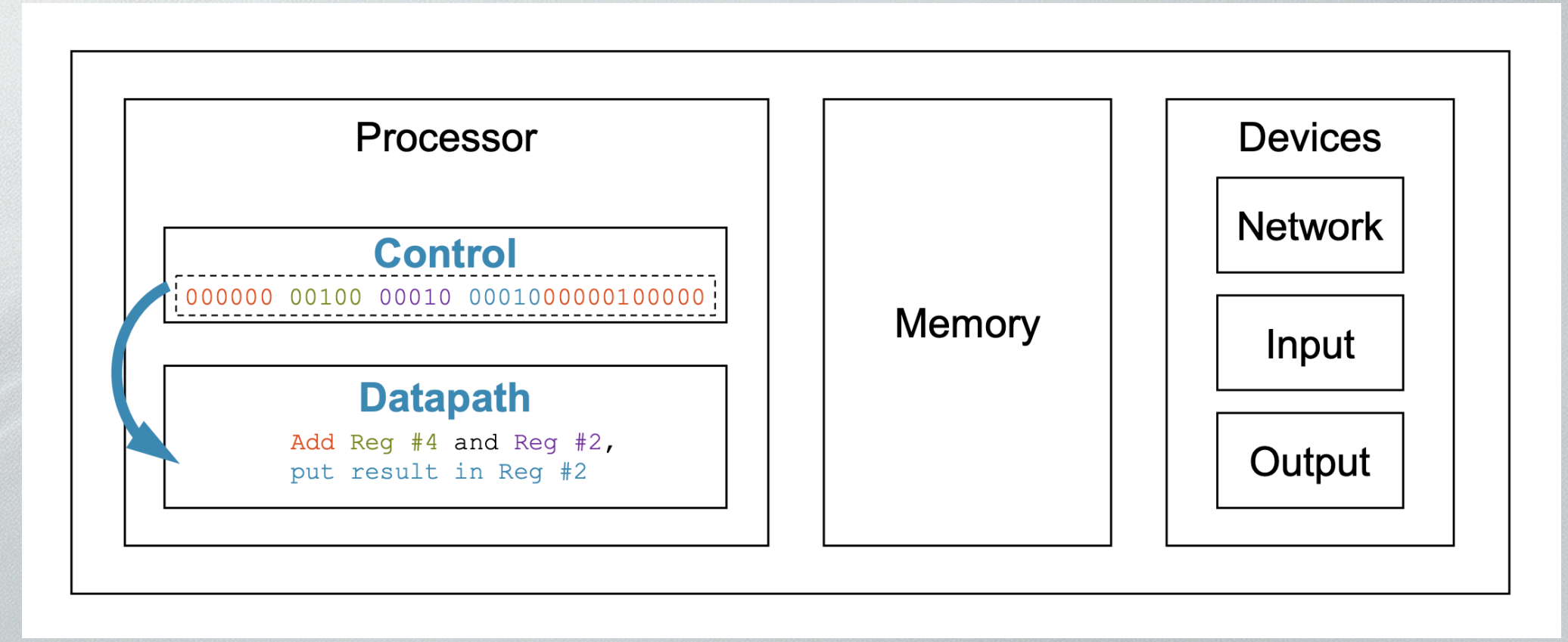
Code Stored in Memory

Under the Covers - The Hardware Components



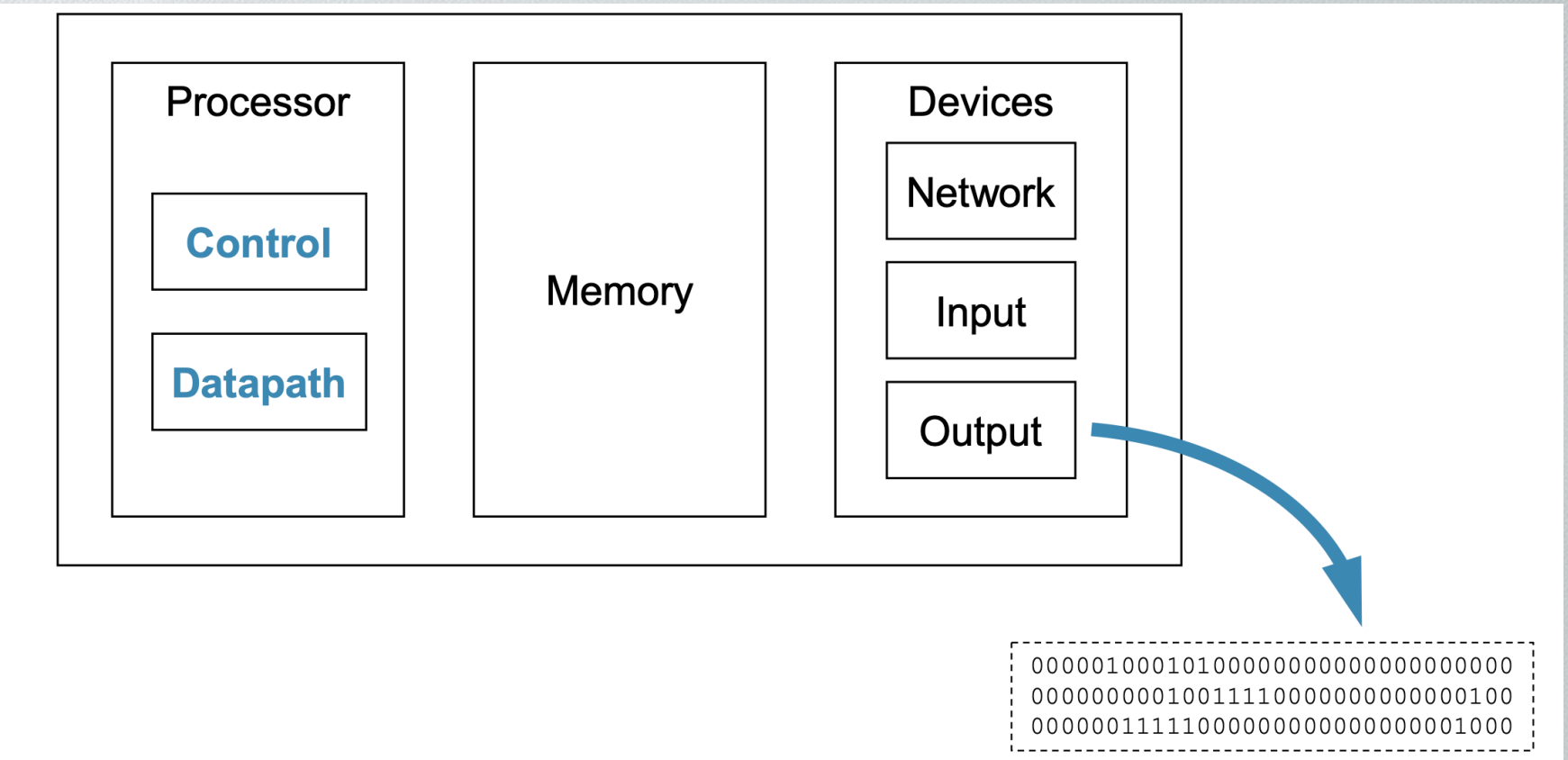
Code Fetch from Memory

Under the Covers – The Hardware Components



Control decodes the instruction to determine what to execute
Datapath executes the instruction as directed by control

Under the Covers - The Hardware Components



Data Output to Device

Technologies for Building Processors and Memory



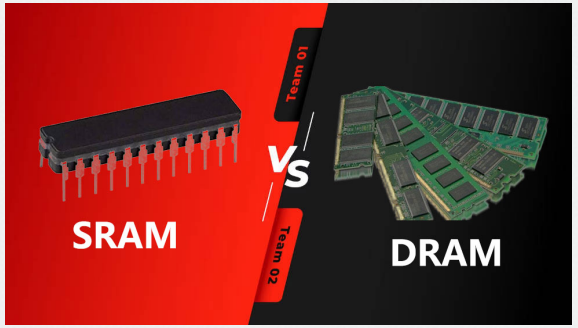
Transistors & VLSI

Transistors: The fundamental on/off switches of digital logic.
Very large-scale integrated (VLSI) circuit: A device containing hundreds of thousands to millions of transistors.



IC Manufacturing

Wafer: A silicon slice no more than 0.1 inches thick where hundreds of chips are fabricated.
Yield: The percentage of good (functional) chips from the total number of chips on the wafer.



Memory Technologies

SRAM/DRAM: Fast, costly cache vs. dense, affordable main memory.
Non-Volatile: Flash memory preserves data without power.

Performance

What is Performance? Defined as **1 / Execution Time**, it measures how quickly a task is completed. A computer is faster if it finishes the same work in less time. For example, if Computer A takes 10s and Computer B takes 15s for the same task, A is **1.5 times faster** (15s / 10s) than B.

Response Time (Execution time)

The total time required for the computer to complete a task, including disk accesses, memory accesses, I/O activities, operating system overhead, CPU execution time, etc.

Throughput (Bandwidth)

The number of tasks completed per unit time.

The Performance Equation

CPU Time = Instruction Count × Cycles per Instruction (CPI) × Clock Cycle Time

Instruction Count

The total number of instructions executed by the program.

Cycles per Instruction (CPI)

The average number of clock cycles required to execute a single instruction.

Clock Cycle Time

The duration for one clock period, usually of the processor clock, which runs at a constant rate.

To improve overall system performance, we must reduce the value of one or more of these three factors — there are no shortcuts!

The Power Wall & The Shift to Multiprocessors

The Power Wall

The physical limit where increasing clock rate leads to unsustainable heat and power consumption. Chips could no longer be cooled efficiently.



From Uniprocessors to Multiprocessors

Introduce parallelism to process with multiple cores. Performance gains from dividing tasks across cores, but requires software designed for parallel execution.

